

Lab 5 Reflection CNN chihuahua or muffin

1. CNN Architecture

The development of the ChihuahuaMuffinCNN architecture using PyTorch further highlights the significant benefits of using convolutional layers for image data compared to the simple multi-layer perceptron architecture from the previous workshop. Instead of turning the 224×224 RGB image into a long one-dimensional vector, thus losing the spatial arrangement and creating many parameters in the process, the convolutional neural network uses sliding 3×3 filters, rectified linear unit activation functions, and max pooling operations. The architecture ensures that the neural network can maintain pixel connectivity and spatial hierarchies natively. It was proven by LeCun et al. (1998) that the use of local receptive fields and weight sharing significantly decreases the model's complexity, as it allows analyzing only parts of the image, thus learning local features like edges or corners irrespective of their position in the image.

2. Model Performance

Performing the model with the prescribed 10 epochs of training led to a reliable and consistent convergence pattern. The initial accuracy of the training process was recorded at 59.17%, with a higher training loss of 3.6274 at Epoch 1, followed by an increasing trend up to 100% maximum validation accuracy at Epoch 7, and finally validation accuracy at 90% at Epoch 10. An analysis of the validation performance and plots related to the same provides an insight into the classification problem faced by the model, i.e., it misclassifies some cases where there is muffin with golden texture and rounded edges, resembling the wrinkled fur and face of a Chihuahua. According to Goodfellow, Bengio, and Courville (2016), these kinds of semantic mix-ups happen because low-level edge and texture maps trigger false positives when visual features overlap across distinct semantic categories, meaning the network relies heavily on higher-level contextual cues to separate look-alike textures.

3. Comparison

When we compare this convolutional neural network with the conventional neural network presented in the previous workshop, there are considerable differences in terms of accuracy and handling of parameters. The classical dense neural network faced issues related to the curse of dimensionality, because converting an entire colorful image into a vector results in more than 150,000 parameters that cause overfitting and slow learning. On the other hand, CNN was able to achieve a reliable validation accuracy of around 90%, through applying filters on localized patches of images. While doing forward and backward passes over the convolutional features requires additional computation for each epoch, the sharing of parameters helps the model learn better gradients and faster features. This is consistent with the study conducted by Krizhevsky et al., (2012), on how convolutional deep networks handle the curse of dimensionality through local connectivity.

4. Challenges and Solutions

I definitely ran into a few annoying roadblocks during the lab setup, like dealing with a package dependency mismatch where upgraded PyTorch binaries clashed with our pinned torchvision version, plus a quick notebook metadata bug with the step-tracking cell. Fixing it meant stopping to clean up version requirements (`torch==2.14.0` and `torchvision==0.29.0`) and making sure our parameter strings matched what the notebook expected. Honestly, dealing with environment errors and dependency hell feels pretty standard for real-world ML work—debugging data pipelines always seems to eat up a chunk of time before you can even get to the actual training.

5. Real-World Applications

Moving beyond the use of this technology on internet memes in order to classify images opens up a wide range of possible applications in different industries. The fact that this type of convolutional pipeline can be used to create automatic quality control systems for the identification of defective parts in the manufacturing process, for the assistance of medical imaging experts in identifying structural abnormalities in X-rays, and even in providing important perceptual information to autonomous robotics is made possible because of a single key factor.

6. Ethical Considerations

Developing models for image classifiers also requires taking into account the moral considerations that come with the use of artificial intelligence in real-world applications. With the limited and unvaried training set, the efficiency of the classifier reduces considerably, and it becomes highly biased when faced with real-world variations. Furthermore, application of machine vision in such critical areas as security and medicine implies serious real-world consequences of the errors in the system. It is important to remember that the development of a model does not involve only striving for accuracy but also taking into consideration how to fail safely.

References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Krizhevsky, A., Sutskever, A., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.